

MIPS Architecture Design

Course Name: T225-EGC 121-SecA, Computer Architecture

Anamitra Basu (BC2025008)
Aaronya Chakraborty (IC2025003)
Aatraya Mukherjee (IC2025004)

Introduction

MIPS (Microprocessor without Interlocked Pipelined Stages) is a family of reduced instruction set computers (RISC) Instruction Set Architectures (ISA) developed by MIPS Computer Systems, now MIPS Technologies, based in the U.S.

There are multiple versions of MIPS, including MIPS I, II, III, IV, and V, as well as five releases of MIPS32/64 (for 32- and 64-bit implementations, respectively). The early MIPS architectures were 32-bit; 64-bit versions were developed later.

Computer Architecture courses in universities and technical schools often study the MIPS architecture. The architecture greatly influenced later RISC architectures such as Alpha. In March 2021, MIPS announced that the development of the MIPS architecture had ended as the company is making the transition to RISC-V.

Processor

We have implemented three programs in MIPS assembly, namely : Selection Sort, Bubble Sort and Binary Search. We have coded the processor in Python, implementing 16 MIPS commands/instructions, including syscalls.

We have used the Mars4_5 compiler to compile the MIPS assembly code and have stored the binary text and data in separate text files. Our processor reads these files during execution. We have assumed the base address of the instructions as 0x00400000 and data as 0x10010000 respectively.

We have divided the processor into mainly three components or classes namely : i) Register (ii) Memory and (iii) CPU. The CPU class contains the functions to trace the 5 stages of MIPS architecture for any instruction namely: IF, ID, EX, MEM, WB. The file containing the processor is named Processor.py.

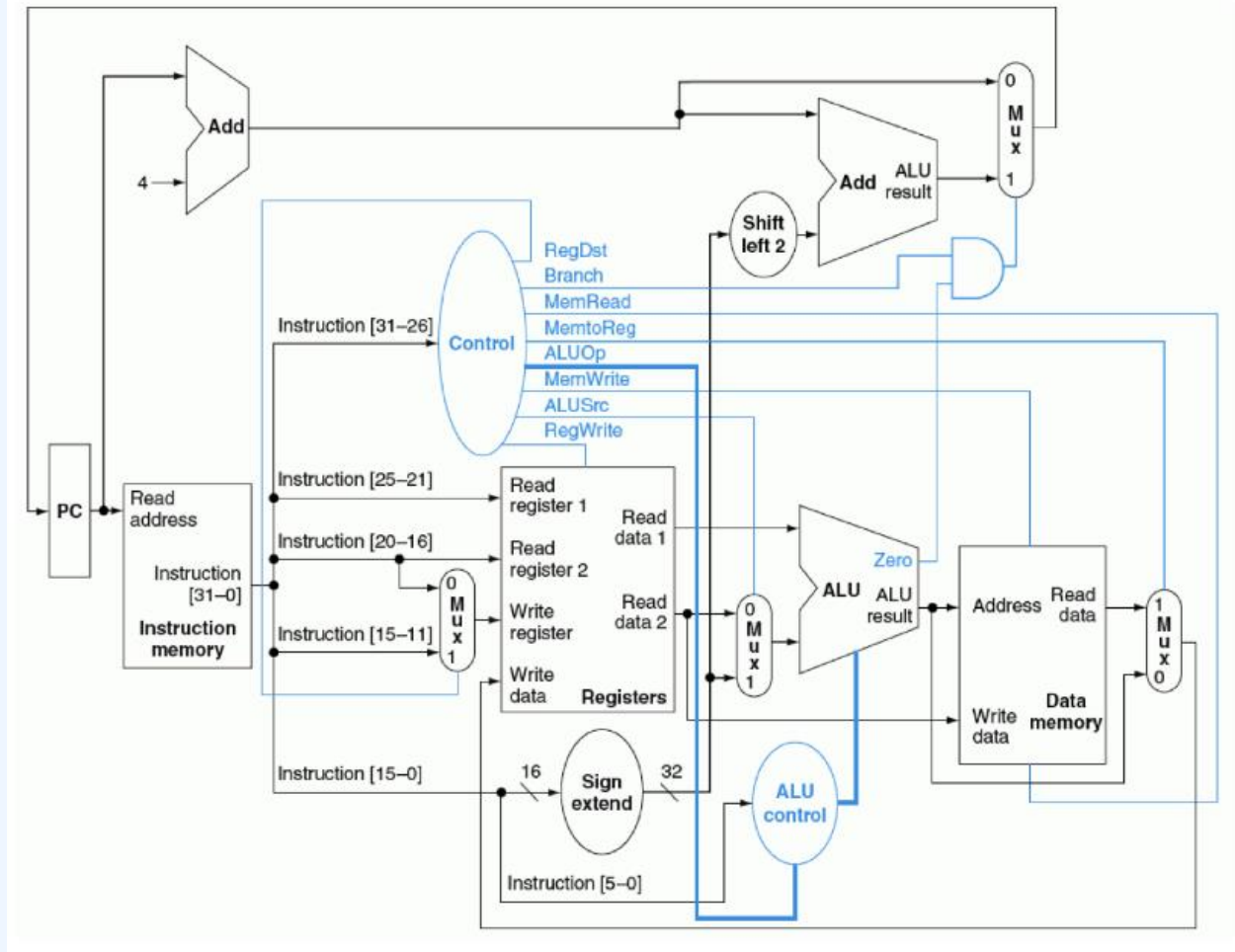


Figure : MIPS Architecture

Selection Sort

This program takes a sequence of n integers and sorts the array in ascending order.

Best case time complexity- $O(n^2)$

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    int arr[5] = {7, 3, 9, 1, 5};

    for(int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for(int j = i + 1; j < n; j++) {
            if(arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    return 0;
}
```

Binary Search

This program takes a sorted array and a target element. It searches for the target in the array and returns the index (0 indexed) if found, otherwise returns -1.

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    vector<int> a={3, 4, 7, 78 , 100, 121, 256};
    int k=3,ans=-1;
    int r=5,l=0;
    while(l<=r)
    {
        int mid = (l+r)/2;
        if(a[mid]==k)
        {
            ans=mid;
            break;
        }
        else if(a[mid]<k)
        {
            l=mid+1;
        }
        else
        {
            r=mid-1;
        }
    }
    //ans contains the answer
}
```

Bubble Sort

This program takes a sequence of n integers and sorts the array in ascending order.

Best case time complexity – $O(n)$

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    int arr[5] = {8, 2, 9, 1, 9};

    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    return 0;
}
```

Definitions of Components used

1. OPCODE DICTIONARY

The Opcode Dictionary is responsible for instruction decoding within the processor. It maps each MIPS instruction mnemonic to its corresponding opcode, function field (for R-type instructions), and associated control signals. In this implementation, two separate dictionaries are maintained: one for R-type instructions and another for I-type and J-type instructions. During the Instruction Decode (ID) stage, the opcode and function bits are extracted from the 32-bit instruction and matched with entries in these dictionaries. Once a match is found, the corresponding control signals are activated. This mechanism enables structured and accurate instruction execution.

2. REGISTER CLASS

The Register class simulates the MIPS register file and maintains the processor state. It includes storage for general-purpose registers such as \$t0–\$t9, \$s0–\$s7, \$a0–\$a3, \$v0–\$v1, \$at, and \$ra. The program counter (PC) is also maintained within this class and is initialized to the base address of the text segment. The \$zero register is hardwired to always return zero. Register values are read during the Execute stage and updated during the Write Back stage. This class ensures proper storage and retrieval of intermediate and final computational results.

3. MEMORY CLASS

The Memory class simulates both instruction memory and data memory using external files. Instruction memory stores 32-bit machine code instructions, while data memory stores 32-bit data words. Address translation is performed relative to predefined base addresses for text and data segments. The class provides methods to read and write instructions and data in binary format. All values are stored as 32-bit representations to maintain architectural consistency. This implementation follows a Harvard-style separation of instruction and data memory.

4. ARITHMETIC LOGIC UNIT (ALU)

The Arithmetic Logic Unit (ALU) performs all arithmetic and logical operations during the Execute (EX) stage of the processor. It receives operands either from registers or immediate fields depending on control signals. The ALU supports operations such as addition, subtraction, logical AND, logical OR, shift operations, and set-less-than comparisons. It also calculates effective addresses for load and store instructions. For branch instructions, it performs comparisons to determine whether a branch condition is satisfied. The ALU output is forwarded to either the memory stage or the write-back stage as required.

5. CONTROL DICTIONARY

The Control Dictionary defines the control signal configuration for each supported instruction. It specifies signals such as RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, Jump, and ALUOp. These signals determine how data flows through the datapath during execution. During the Instruction Decode stage, the appropriate set of control signals is selected based on the instruction opcode. This structured approach simplifies control logic implementation. The Control Dictionary effectively represents the processor's control unit behavior.

6. CONTROL LINES

Control lines are binary signals generated during the Decode stage to manage processor operations. These signals control multiplexers, memory access, register writing, and program counter updates. For example, RegWrite enables register updates, while MemRead and MemWrite manage data memory access. The Branch and Jump signals modify control flow by updating the program counter when necessary. ALUSrc and RegDst determine operand selection and destination register selection. Together, these control lines coordinate the functioning of all five stages: IF, ID, EX, MEM, and WB.

Processor

Processor code:

```
1  TEXT_BASE = 0x00400000
2  DATA_BASE = 0x10010000
3
4
5  R_TYPE_OPCODE = {
6
7      "add": {"opcode": "000000", "funct": "100000",
8              "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
9              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
10             "Branch": 0, "Jump": 0, "ALUOp": "10"},
11
12      "sub": {"opcode": "000000", "funct": "100010",
13              "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
14              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
15              "Branch": 0, "Jump": 0, "ALUOp": "10"},
16
17      "and": {"opcode": "000000", "funct": "100100",
18              "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
19              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
20              "Branch": 0, "Jump": 0, "ALUOp": "10"},
21
22      "or": {"opcode": "000000", "funct": "100101",
23             "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
24             "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
25             "Branch": 0, "Jump": 0, "ALUOp": "10"},
26
27      "slt": {"opcode": "000000", "funct": "101010",
28              "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
29              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
30              "Branch": 0, "Jump": 0, "ALUOp": "10"},
31
32      "sll": {"opcode": "000000", "funct": "000000",
33              "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
34              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
35              "Branch": 0, "Jump": 0, "ALUOp": "10"},
36
37      "srl": {"opcode": "000000", "funct": "000010",
38              "RegDst": 1, "ALUSrc": 0, "MemtoReg": 0,
39              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
40              "Branch": 0, "Jump": 0, "ALUOp": "10"},
41
42      "syscall": {"opcode": "000000", "funct": "001100",
43                  "RegDst": None, "ALUSrc": None, "MemtoReg": None,
44                  "RegWrite": 0, "MemRead": 0, "MemWrite": 0,
45                  "Branch": 0, "Jump": 0, "ALUOp": None}
46
47  }
48
49
50  OPCODE_FUNCT = {
51      "addi": {"opcode": "001000", "funct": None,
52              "RegDst": 0, "ALUSrc": 1, "MemtoReg": 0,
53              "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
54              "Branch": 0, "Jump": 0, "ALUOp": "00"},
55
56      "andi": {"opcode": "001100", "funct": None,
```



```

55
56     "ori": {"opcode": "001101", "funct": None,
57            "RegDst": 0, "ALUSrc": 1, "MemtoReg": 0,
58            "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
59            "Branch": 0, "Jump": 0, "ALUOp": "11"},
60
61     "lui": {"opcode": "001111", "funct": None,
62            "RegDst": 0, "ALUSrc": 1, "MemtoReg": 0,
63            "RegWrite": 1, "MemRead": 0, "MemWrite": 0,
64            "Branch": 0, "Jump": 0, "ALUOp": "LUI"},
65
66
67     "lw": {"opcode": "100011", "funct": None,
68           "RegDst": 0, "ALUSrc": 1, "MemtoReg": 1,
69           "RegWrite": 1, "MemRead": 1, "MemWrite": 0,
70           "Branch": 0, "Jump": 0, "ALUOp": "00"},
71
72     "sw": {"opcode": "101011", "funct": None,
73           "RegDst": None, "ALUSrc": 1, "MemtoReg": None,
74           "RegWrite": 0, "MemRead": 0, "MemWrite": 1,
75           "Branch": 0, "Jump": 0, "ALUOp": "00"},
76
77
78     "beq": {"opcode": "000100", "funct": None,
79            "RegDst": None, "ALUSrc": 0, "MemtoReg": None,
80            "RegWrite": 0, "MemRead": 0, "MemWrite": 0,
81            "Branch": 1, "Jump": 0, "ALUOp": "01"},
82
83     "bne": {"opcode": "000101", "funct": None,
84            "RegDst": None, "ALUSrc": 0, "MemtoReg": None,
85            "RegWrite": 0, "MemRead": 0, "MemWrite": 0,
86            "Branch": 1, "Jump": 0, "ALUOp": "01"},
87
88
89     "j": {"opcode": "000010", "funct": None,
90           "RegDst": None, "ALUSrc": None, "MemtoReg": None,
91           "RegWrite": 0, "MemRead": 0, "MemWrite": 0,
92           "Branch": 0, "Jump": 1, "ALUOp": None}
93 }
94
95
96
97
98
99
100 class Register:
101     def __init__(self):
102         self.t = [0] * 10
103         self.s = [0] * 8
104         self.a = [0] * 4
105
106         self.pc = TEXT_BASE
107
108         self.zero = 0

```

```

107         self.instr = instr
108         self.zero = 0
109         self.at = 0
110         self.ra = 0
111         self.v0 = 0
112         self.v1 = 0
113
114
115
116 class Memory:
117
118     def __init__(self, file_instr, file_data):
119         self.data_memory = file_data
120         self.instr_memory = file_instr
121
122
123     def read_data(self, addr):
124         with open(self.data_memory, "r") as f:
125             for i, line in enumerate(f):
126                 if i == addr:
127                     return line.strip()
128         return ""
129
130
131     def read_instr(self, addr):
132         with open(self.instr_memory, "r") as f:
133             for i, line in enumerate(f):
134                 if i == addr:
135                     return line.strip()
136         return ""
137
138     def write_data(self, addr, value):
139         value &= ((1 << 32) - 1)
140
141         with open(self.data_memory, "r") as f:
142             lines = f.readlines()
143
144             while len(lines) <= addr:
145                 lines.append("0".zfill(32) + "\n")
146
147             lines[addr] = format(value, "032b") + "\n"
148
149             with open(self.data_memory, "w") as f:
150                 f.writelines(lines)
151
152
153     def write_instr(self, addr, value):
154         value &= ((1 << 32) - 1)
155
156         with open(self.instr_memory, "r") as f:
157             lines = f.readlines()
158
159             while len(lines) <= addr:
160                 lines.append("0".zfill(32) + "\n")
161

```

```

159         write ten(lines) <- addr.
160
161
162         lines[addr] = format(value, "032b") + "\n"
163
164         with open(self.instr_memory, "w") as f:
165             f.writelines(lines)
166
167
168
169 class CPU:
170     def __init__(self, instr_file, data_file):
171         self.mem = Memory(instr_file, data_file)
172         self.reg = Register()
173         self.running = True
174         self.instr=""
175         self.data=""
176         self.opcode=""
177         self.rs=""
178         self.rt=""
179         self.rd=""
180         self.shamt=""
181         self.func=""
182         self.imme = 0
183         self.jaddr = 0
184         self.ctrl = None
185         self.current_instr = ""
186         self.alu_result = 0
187         self.mem_data = 0
188         self.cycle=0
189
190
191     def print_reg(self):
192         print("\n=====")
193         print(f"Cycle: {self.cycle}")
194         print(f"PC: {hex(self.reg.pc)}")
195
196         print("Registers:")
197
198         print("---- $a registers ----")
199         for i in range(4):
200             if self.reg.a[i]!=0:
201                 print(f"$a{i} = {self.reg.a[i]}")
202
203         print("---- $t registers ----")
204         for i in range(10):
205             if self.reg.t[i]!=0:
206                 print(f"$t{i} = {self.reg.t[i]}")
207
208         print("---- $s registers ----")
209         for i in range(8):
210             if self.reg.s[i]!=0:
211                 print(f"$s{i} = {self.reg.s[i]}")
212
213         print("=====\\n")

```

```

214
215     def IF_stage(self):
216         self.instr = self.mem.read_instr((self.reg.pc-TEXT_BASE) // 4)
217         if not self.instr: # Stop if no more instructions
218             self.running = False
219             return
220         self.reg.pc += 4
221
222
223     def ID_stage(self):
224         self.ctrl = None
225         self.current_instr = None
226
227         self.opcode = self.instr[0:6]
228         self.rs = int(self.instr[6:11],2)
229         self.rt = int(self.instr[11:16],2)
230
231         if self.opcode == "000000":
232             self.rd = int(self.instr[16:21],2)
233             self.shamt = int(self.instr[21:26],2)
234             self.func = self.instr[26:32]
235
236
237             for name,signals in R_TYPE_OPCODE.items():
238                 if signals["funct"] == self.func:
239                     self.ctrl = signals.copy()
240                     self.current_instr = name
241                     break
242
243
244         elif self.opcode in ["000010"] :
245             self.jaddr = int(self.instr[6:32],2)
246             self.current_instr = "j"
247
248             self.ctrl = {"opcode":"000010","funct":None,
249                 "RegDst":None,"ALUSrc":None,"MemtoReg":None,
250                 "RegWrite":0,"MemRead":0,"MemWrite":0,
251                 "Branch":0,"Jump":1,"ALUOp":None}
252
253         else:
254             self.rd = None
255             raw_imm = int(self.instr[16:32], 2)
256             if raw_imm & 0x8000:
257                 self.imme = raw_imm - 0x10000
258             else:
259                 self.imme = raw_imm
260
261             for name, signals in OPCODE_FUNCT.items():
262                 if signals["opcode"] == self.opcode:
263                     self.ctrl = signals.copy()
264                     self.current_instr = name
265                     break
266

```

```

266
267     if self.ctrl is None:
268         print("ERROR: Unknown instruction")
269         print("Opcode:", self.opcode)
270         self.running = False
271
272
273     def get_register_val(self, register_idx):
274         if register_idx == 0: return 0 # $zero
275         if register_idx == 1: return self.reg.at
276         if register_idx == 2: return self.reg.v0
277         if register_idx == 3: return self.reg.v1
278         if 4 <= register_idx <= 7: return self.reg.a[register_idx - 4]
279         if 8 <= register_idx <= 15: return self.reg.t[register_idx - 8]
280         if 16 <= register_idx <= 23: return self.reg.s[register_idx - 16]
281         if 24 <= register_idx <= 25: return self.reg.t[register_idx - 16] # t8-t9
282         if register_idx == 31: return self.reg.ra
283         return 0
284
285     def EX_stage(self):
286
287         # 1. Fetch Operands
288         val_at_rs = self.get_register_val(self.rs)
289         val_at_rt = self.get_register_val(self.rt)
290
291         # 2. ALU Mux: Choose between Register (0) or Immediate (1)
292         operand_b = val_at_rt # default
293
294         if self.ctrl and self.ctrl["ALUSrc"] is not None:
295             operand_b = val_at_rt if self.ctrl["ALUSrc"] == 0 else self.imme
296
297         # 3. ALU Operations
298         if self.current_instr == "add":
299             self.alu_result = val_at_rs + operand_b
300
301         elif self.current_instr == "sub":
302             self.alu_result = val_at_rs - operand_b
303
304         elif self.current_instr == "and":
305             self.alu_result = val_at_rs & operand_b
306
307         elif self.current_instr == "or":
308             self.alu_result = val_at_rs | operand_b
309
310         elif self.current_instr == "sll":
311             # Shift rt, not rs
312             self.alu_result = val_at_rt << self.shamt
313
314
315

```

```

315
316     elif self.current_instr == "srl":
317         # Shift rt, not rs
318         self.alu_result = val_at_rt >> self.shamt
319
320     elif self.current_instr == "slt":
321         self.alu_result = 1 if val_at_rs < operand_b else 0
322
323     elif self.current_instr == "addi":
324         self.alu_result = val_at_rs + operand_b
325
326     elif self.current_instr == "ori":
327         self.alu_result = val_at_rs | operand_b
328
329     elif self.current_instr == "lui":
330         self.alu_result = self.imme << 16
331
332     # 4. Memory Address Calculation (Base + Offset)
333     elif self.current_instr in ["lw", "sw"]:
334         self.alu_result = val_at_rs + self.imme
335
336     # 5. Branch Operations
337     # Target = PC + (Offset * 4). Note: PC was already +4 in IF_stage.
338     elif self.current_instr == "beq":
339         if val_at_rs == val_at_rt:
340             self.reg.pc += (self.imme * 4)
341
342     elif self.current_instr == "bne":
343         if val_at_rs != val_at_rt:
344             self.reg.pc += (self.imme * 4)
345
346
347     elif self.current_instr == "j":
348         pc_upper = (self.reg.pc & 0xF0000000)
349         pc_upper = (self.reg.pc & 0xF0000000)
350         self.reg.pc = pc_upper | (self.jaddr * 4)
351 # Concatenate PC upper bits with jaddr shifted left by 2
352
353     elif self.current_instr == "syscall":
354         if self.reg.v0 == 10:
355             self.running = False
356
357
358
359     def MEM_stage(self):
360
361         if not self.ctrl:
362             return
363
364         if self.ctrl["MemRead"] == 1 or self.ctrl["MemWrite"] == 1:
365             print("ALU_RESULT:", hex(self.alu_result))
366             address = (self.alu_result - DATA_BASE)//4
367

```

```

368         if self.ctrl["MemRead"] == 1:
369             raw_data = self.mem.read_data(address)
370             if raw_data:
371                 val = int(raw_data, 2)
372                 # Check if the sign bit (bit 31) is 1
373                 self.mem_data = val - 0x100000000 if val & 0x80000000 else val
374             else:
375                 self.mem_data = 0
376
377         if self.ctrl["MemWrite"] == 1:
378             val_at_rt = self.get_register_val(self.rt)
379
380             self.mem.write_data(address, val_at_rt)
381
382
383     def WB_stage(self):
384         if not self.ctrl:
385             return
386         if self.ctrl["RegWrite"] == 0:
387             return
388
389         write_value = self.mem_data if self.ctrl["MemtoReg"] == 1 else self.alu_result
390         dest_reg = self.rd if self.ctrl["RegDst"] == 1 else self.rt
391
392         if dest_reg == 0: return
393
394         # Mapping logic (must match get_register_val)
395         if dest_reg == 2: self.reg.v0 = write_value
396         elif dest_reg == 1: self.reg.at = write_value
397         elif dest_reg == 3: self.reg.v1 = write_value
398         elif 4 <= dest_reg <= 7:
399             self.reg.a[dest_reg - 4] = write_value
400         elif 8 <= dest_reg <= 15: self.reg.t[dest_reg - 8] = write_value
401         elif 16 <= dest_reg <= 23: self.reg.s[dest_reg - 16] = write_value
402         elif 24 <= dest_reg <= 25: self.reg.t[dest_reg - 16] = write_value
403         elif dest_reg == 31: self.reg.ra = write_value
404
405
406     def run(self):
407         while self.running:
408             self.cycle += 1
409
410             self.IF_stage()
411             if not self.running: break
412             self.ID_stage()
413             self.EX_stage()
414             self.MEM_stage()
415             self.WB_stage()
416
417             if self.cycle % 10 == 0:
418                 self.print_reg()
419
420         # =====
421
422
423     def get_files():
424         file_instr = input("Enter instruction filename: ")
425         file_data = input("Enter data filename: ")
426         return (file_instr, file_data)
427
428     file_list = get_files()
429     file_instr = file_list[0]
430     file_data = file_list[1]
431
432
433     cpu = CPU(file_instr, file_data)
434     cpu.run()

```

MIPS Assembly Codes:

1.Selection Sort


```

1  .data
2  A: .word 7, 3, 9, 1, 5
3  n: .word 5
4
5  .text
6  .globl main
7
8  main:
9
10     lui $a0, 0x1001
11     ori $a0, $a0, 0x0000
12
13     lui $t9, 0x1001
14     ori $t9, $t9, 0x0014
15     lw  $a1, 0($t9)
16
17     addi $t0, $zero, 0      # i = 0
18
19 outer_loop:
20     addi $t7, $a1, -1
21     slt  $t8, $t0, $t7
22     beq  $t8, $zero, exit
23
24     add  $t1, $t0, $zero
25     addi $t2, $t0, 1
26
27 inner_loop:
28     slt  $t8, $t2, $a1
29     beq  $t8, $zero, swap
30
31     sll  $t3, $t2, 2
32     add  $t3, $t3, $a0
33     lw   $t4, 0($t3)
34
35     sll  $t5, $t1, 2
36     add  $t5, $t5, $a0
37     lw   $t6, 0($t5)
38
39     slt  $t8, $t4, $t6
40     beq  $t8, $zero, skip_update

```

```

41
42     add $t1, $t2, $zero
43
44     skip_update:
45         addi $t2, $t2, 1
46         j inner_loop
47
48     swap:
49         beq $t1, $t0, next_i
50
51         sll $t3, $t0, 2
52         add $t3, $t3, $a0
53         lw  $t4, 0($t3)
54
55         sll $t5, $t1, 2
56         add $t5, $t5, $a0
57         lw  $t6, 0($t5)
58
59         sw  $t6, 0($t3)
60         sw  $t4, 0($t5)
61
62     next_i:
63         addi $t0, $t0, 1
64         j outer_loop
65
66     exit:
67         addi $v0, $zero, 10
68         syscall

```

2. Binary Search

```

1  .data
2      a: .word 3, 4, 7, 78, 100, 121, 256
3      k_val: .word 4
4
5  .text
6  .globl main
7  main:
8      la $s0, a          #Get actual base address
9      addi $s1, $zero, 0  #l = 0
10     addi $s2, $zero, 6  #r = 5 (size - 1)
11     lw $s3, k_val       #Better to load k from memory too
12
13 loop:
14     #Check if l > r
15     slt $t0, $s2, $s1    #If r < l, $t0 = 1
16     bne $t0, $zero, fail #If r < l, jump to fail
17
18     # mid = (l + r)/2
19     add $t1, $s1, $s2    #t1 = l + r
20     srl $s4, $t1, 1      #mid = (l + r) / 2
21
22     #Load a[mid] and check if a[mid] < k
23     #MIPS requires calculating byte offset (index * 4)
24     sll $t2, $s4, 2      #t2 = mid * 4
25     add $t2, $t2, $s0    #t2 = address of a[mid]
26     lw $s5, 0($t2)       #s5 = a[mid]
27
28     slt $t3, $s5, $s3     #If a[mid] < k, $t3 = 1
29     bne $t3, $zero, l_up  #If a[mid] < k, go to l = mid + 1
30
31     #Check if a[mid] == k or a[mid] > k
32     beq $s5, $s3, found   #If a[mid] == k, go to found
33
34     # r = mid - 1 (If a[mid] > k)
35     addi $s2, $s4, -1     #r = mid - 1
36     j loop
37
38 l_up:
39     #l = mid + 1
40     addi $s1, $s4, 1      #l = mid + 1
41     j loop

```

```

42
43 found:
44     #Store result and stop execution
45     add $s6, $s4, $zero   # Store mid in result register
46     j done
47
48 fail:
49     addi $s6, $zero, -1
50     # Result = -1 (not found)
51
52 done:
53     # tells the simulator to stop

```

3. Bubble Sort

```

1  .data
2  A:  .word 8, 2, 9, 1, 9
3  n:  .word 5
4
5  .text
6  .globl main
7
8  main:
9
10     lui  $a0, 0x1001
11     ori  $a0, $a0, 0x0000
12
13     lui  $t9, 0x1001
14     ori  $t9, $t9, 0x0014
15     lw   $a1, 0($t9)
16
17     addi $t0, $zero, 0      #i = 0
18
19 outer_loop:
20     addi $t7, $a1, -1      #n-1
21     slt  $t8, $t0, $t7
22     beq  $t8, $zero, exit  #if i >= n-1 exit
23
24     addi $t1, $zero, 0      #j = 0
25
26 inner_loop:
27     sub  $t6, $a1, $t0      #n - i
28     addi $t6, $t6, -1       #n - i - 1
29
30     slt  $t8, $t1, $t6
31     beq  $t8, $zero, next_i #if j >= n-i-1 go next i
32
33     # Load A[j]
34     sll  $t2, $t1, 2
35     add  $t2, $t2, $a0
36     lw   $t3, 0($t2)

```

```

37
38     # Load A[j+1]
39     addi $t4, $t1, 1
40     sll $t4, $t4, 2
41     add $t4, $t4, $a0
42     lw $t5, 0($t4)
43
44     # if A[j] > A[j+1]
45     slt $t8, $t5, $t3      #(A[j+1] < A[j])
46     beq $t8, $zero, no_swap
47
48     # swap
49     sw $t5, 0($t2)
50     sw $t3, 0($t4)
51
52 no_swap:
53     addi $t1, $t1, 1
54     j inner_loop
55
56 next_i:
57     addi $t0, $t0, 1
58     j outer_loop
59
60 exit:
61     addi $v0, $zero, 10
62     syscall

```

Execution Results

The below screenshots show the result after running the processor.py , using only the binary files for the instruction and data segment.

Selection Sort

Before Sorting (the data segment):

1		00000000000000000000000000000111
2		0000000000000000000000000000000011
3		00000000000000000000000000000001001
4		00000000000000000000000000000000001
5		00000000000000000000000000000000101
6		00000000000000000000000000000000101
7		00000000000000000000000000000000000
8		00000000000000000000000000000000000
9		00000000000000000000000000000000000
10		00000000000000000000000000000000000

After Sorting(the data segment):

1	00000000000000000000000000000001
2	000000000000000000000000000000001
3	00000000000000000000000000000000101
4	00000000000000000000000000000000111
5	000000000000000000000000000000001001
6	00000000000000000000000000000000101
7	00000000000000000000000000000000000
8	00000000000000000000000000000000000
9	00000000000000000000000000000000000
10	00000000000000000000000000000000000

Figure: Result of Selection Sort.

Binary Search:

We search on the array [3, 4, 7, 78, 100, 121, 256] for the element 4.

\$s6 should contain the index of the element.

```

Enter instruction filename: BS_Text.txt
Enter data filename: BS_Data.txt
ALU_RESULT: 0x1001001c

=====
Cycle: 10
PC: 0x400028
Registers:
---- $a registers ----
---- $t registers ----
$t1 = 6
---- $s registers ----
$s0 = 268500992
$s2 = 6
$s3 = 4
$s4 = 3
=====

ALU_RESULT: 0x1001000c

=====
Cycle: 20
PC: 0x400020
Registers:
---- $a registers ----
---- $t registers ----
$t1 = 6
$t2 = 268501004
---- $s registers ----
$s0 = 268500992
$s2 = 2
$s3 = 4
$s4 = 3
$s5 = 78
=====

ALU_RESULT: 0x10010004

=====
Cycle: 30
PC: 0x40005c
Registers:
---- $a registers ----
---- $t registers ----
$t1 = 2
$t2 = 268500996
---- $s registers ----
$s0 = 268500992
$s2 = 2
$s3 = 4
$s4 = 1
$s5 = 4
$s6 = 1
=====

```

Figure: Result of Binary Search.

Bubble Sort:

Before Sorting(the data segment)

[illegible]

After Sorting(the data segment)

[illegible]

Figure: Result of Bubble Sort

References:

- [https://en.wikipedia.org/wiki/Stanford MIPS](https://en.wikipedia.org/wiki/Stanford_MIPS)
- Computer organization and design MIPS Edition, Hennessey and Patterson (H&P). 5th edition
- Digital design and computer architecture, Harris and Harris.